Decision Tree Challenge

Feature Importance and Categorical Variable Encoding

🌳 Decision Tree Challenge - Feature Importance and Variable Encoding

📋 **What You Need To Do**

- ☐ Fork this repository to create `decTreeChallenge` in your GitHub
- ☐ Clone locally and open in Cursor
- ☐ Write your analysis in this `index.qmd`
- ☐ Render to HTML (`quarto render` or Render button)
- ☐ Push HTML to your repo and enable GitHub Pages
- ☐ Share your site: `https://[your-username].github.io/decTreeChallenge/`

🐍 **Python Environment Setup**

Before rendering, create a fresh virtual environment and install the required packages:

1. Press `Ctrl+Shift+P` to open the Command Palette
2. Type **"Python: Select Interpreter"** and select it
3. Click **"+ Create Virtual Environment..."**
4. Choose **Venv**
5. Select your Python installation (e.g., Python 3.12)
6. Wait for the environment to be created and activated

Then open a terminal and install dependencies:

Windows:

```
py -m pip install jupyter matplotlib pandas scikit-learn
```

Mac:

```
python3 -m pip install jupyter matplotlib pandas scikit-learn
```

⚠️ **AI Partnership Required**

This challenge pushes boundaries intentionally. You'll tackle problems that normally require weeks of study, but with Cursor AI as your partner (and your brain keeping it honest), you can accomplish more than you thought possible.

The new reality: The four stages of competence are Ignorance → Awareness → Learning → Mastery. AI lets us produce Mastery-level work while operating primarily in the Awareness stage. I focus on awareness training, you leverage AI for execution, and together we create outputs that used to require years of dedicated study.

📊 **Grading Criteria**

- **Question 1:** Required. Up to **75%** of the grade.
- **Question 2:** Up to **95%** of the grade.
- **Question 3:** Up to **100%** of the grade (extra 5 points – see warning below Q3).

Your grade depends on narrative clarity and insight, not just correct code.

📝 How to Present Your Analysis

This is an investigative report, not a coding exercise. Present your findings like a professional analyst writing a brief about why proper variable encoding matters in machine learning.

- **Q&A Format:** Each discussion question clearly stated, followed by your answer with analysis and interpretation.
- **Delete All Instructions:** Remove all challenge instructions, setup guides, and grading criteria from your final rendered HTML. The final report should contain only the analysis and your discussion responses.
- **Hide Code:** Tell a narrative and visual story using `echo: false`; the code can be referenced in your GitHub `*.qmd` source file.
- **Concise and Human:** No AI babble. Tell the story of what you discovered, focus on insights, and write like a data science consultant.

What we're looking for: A compelling 1-2 minute read that demonstrates both the power of decision trees for interpretability and the critical importance of proper variable encoding.

💾 Save Your Work Frequently!

Commit often using Source Control (Ctrl+Shift+G). Commit after completing each analysis step, after finishing each discussion question, and before asking AI for help with new code.

The Decision Tree Problem 🎯

> "The most important thing in communication is hearing what isn't said." - Peter Drucker

The Core Problem: Decision trees are often praised for their interpretability and ability to handle both numerical and categorical variables. But what happens when we encode categorical variables as numbers? How does this affect our understanding of feature importance?

What is Feature Importance? In decision trees, feature importance measures how much each variable contributes to reducing impurity (or improving prediction accuracy) across all splits in the tree. It's a key metric for understanding which variables matter most for your predictions.

🎯 The Key Insight: Encoding Matters for Interpretability

The problem: When we encode categorical variables as numerical values (like 1, 2, 3, 4...), decision trees treat them as if they have a meaningful numerical order. This can completely distort our analysis.

The Real-World Context: In real estate, we know that neighborhood quality, house style, and other categorical factors are crucial for predicting home prices. But if we encode these as numbers, we might get misleading insights about which features actually matter most.

The Devastating Reality: Even sophisticated machine learning models can give us completely wrong insights about feature importance if we don't properly encode our variables. A categorical variable that should be among the most important might appear irrelevant, while a numerical variable might appear artificially important.

Let's assume we want to predict house prices and understand which features matter most. The key question is: **How does encoding categorical variables as numbers affect our understanding of feature importance?**

The Ames Housing Dataset 🏠

We are analyzing the Ames Housing dataset which contains detailed information about residential properties sold in Ames, Iowa from 2006 to 2010. This dataset is perfect for our analysis because it contains a categorical variable (like zip code) and numerical variables (like square footage, year built, number of bedrooms).

The Problem: ZipCode as Numerical vs Categorical

Key Question: What happens when we treat zipCode as a numerical variable in a decision tree? How does this affect feature importance interpretation?

The Issue: Zip codes (50010, 50011, 50012, 50013) are categorical variables representing discrete geographic areas, i.e. neighborhoods. When treated as numerical, the tree might split on "zipCode > 50012.5" - which has no meaningful interpretation for house prices. Zip codes are non-ordinal categorical variables meaning they have no inherent order that aids house price prediction (i.e. zip code 99999 is not the priceiest zip code).

Data Loading and Model Building

Python

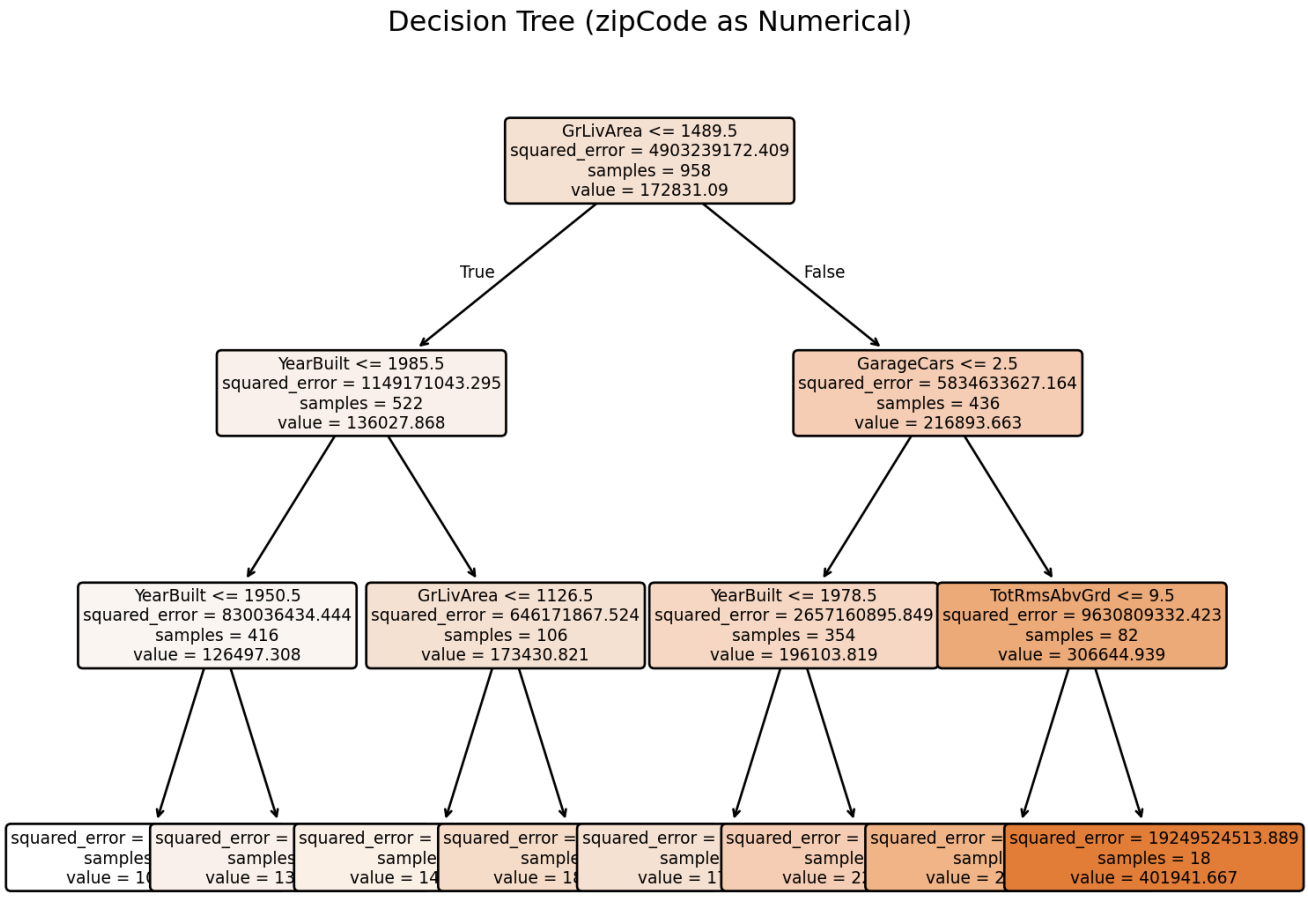
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Load data
sales_data = pd.read_csv("salesPriceData.csv")

# Prepare model data (treating zipCode as numerical)
model_vars = ['SalePrice', 'LotArea', 'YearBuilt', 'GrLivArea', 'FullBath',
              'HalfBath', 'BedroomAbvGr', 'TotRmsAbvGrd', 'GarageCars', 'zipCode']
model_data = sales_data[model_vars].dropna()

# Split data
X = model_data.drop('SalePrice', axis=1)
y = model_data['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123)

# Build decision tree
tree_model = DecisionTreeRegressor(max_depth=3,
                                   min_samples_split=20,
                                   min_samples_leaf=10,
                                   random_state=123)
tree_model.fit(X_train, y_train)

print(f"Model built with {tree_model.get_n_leaves()} terminal nodes")
```

```
Model built with 8 terminal nodes
```

Tree Visualization

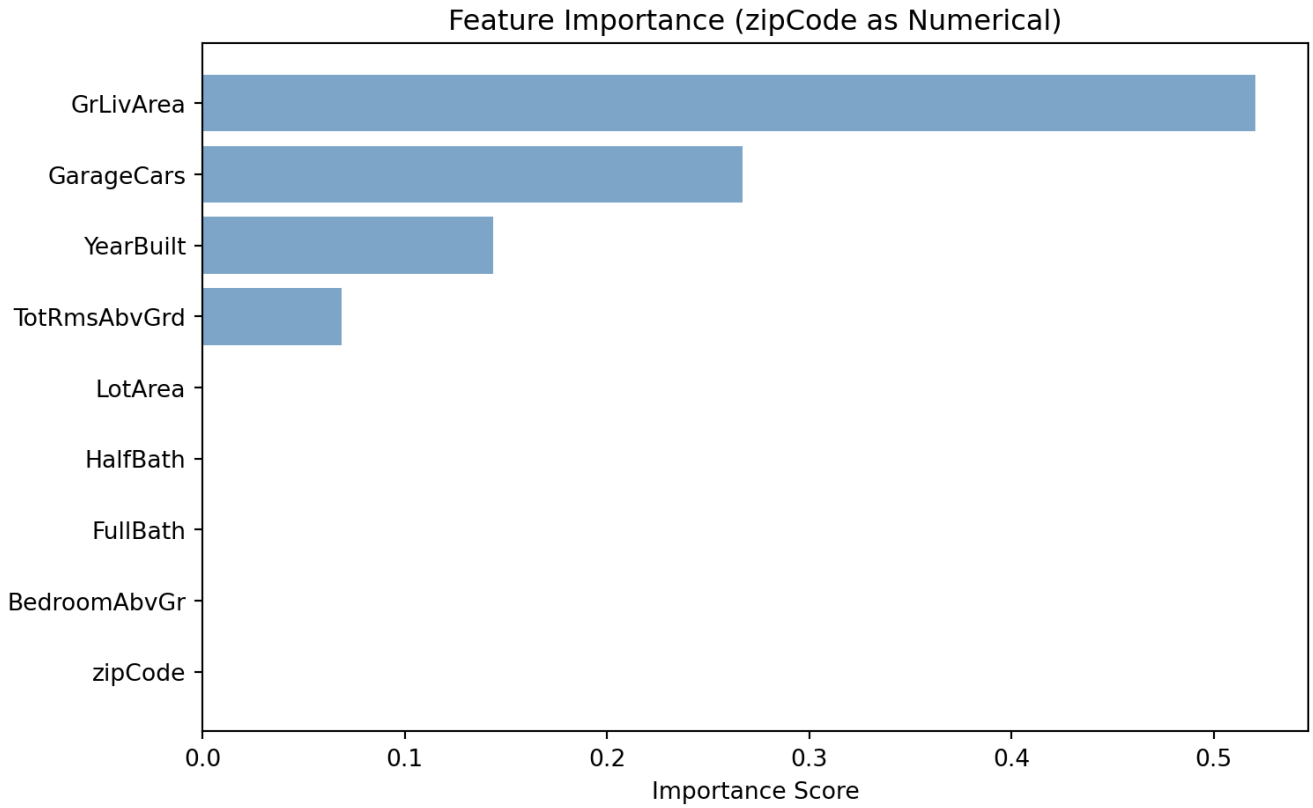
Python

```python
# Visualize tree
plt.figure(figsize=(8, 6))
plot_tree(tree_model,
          feature_names=X_train.columns,
          filled=True,
          rounded=True,
          fontsize=7,
          max_depth=3)
plt.title("Decision Tree (zipCode as Numerical)")
plt.tight_layout()
plt.show()
```



Feature Importance Analysis

Python



Critical Analysis: The Encoding Problem

⚠️ The Problem Revealed

What to note: Our decision tree treated `zipCode` as a numerical variable. This leads to zip code being unimportant. Not surprisingly, because there is no reason to believe allowing splits like "zipCode < 50012.5" should be beneficial for house price prediction. This false coding of a variable creates several problems:

1. **Potentially Meaningless Splits:** A zip code of 50013 is not "greater than" 50012 in any meaningful way for house prices
2. **False Importance:** The algorithm assigns importance to zipCode based on numerical splits rather than categorical distinctions OR the importance of zip code is completely missed as numerical ordering has no inherent relationship to house prices.
3. **Misleading Interpretations:** We might conclude zipCode is not important when our intuition tells us it should be important (listen to your intuition).

The Real Issue: Zip codes are categorical variables representing discrete geographic areas. The numerical values have no inherent order or magnitude relationship to house prices. These must be modelled as categorical variables.

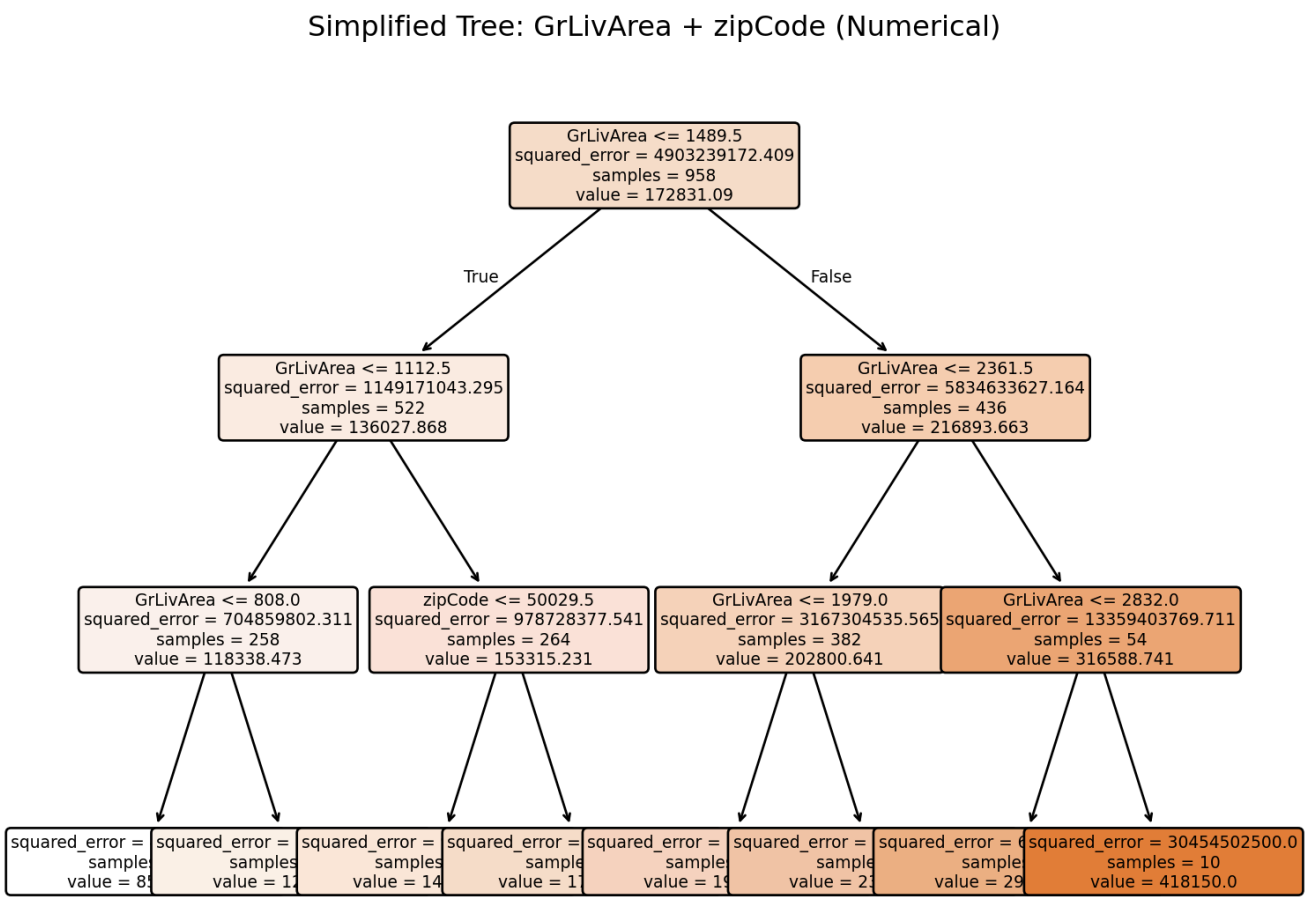
Isolating the Encoding Effect

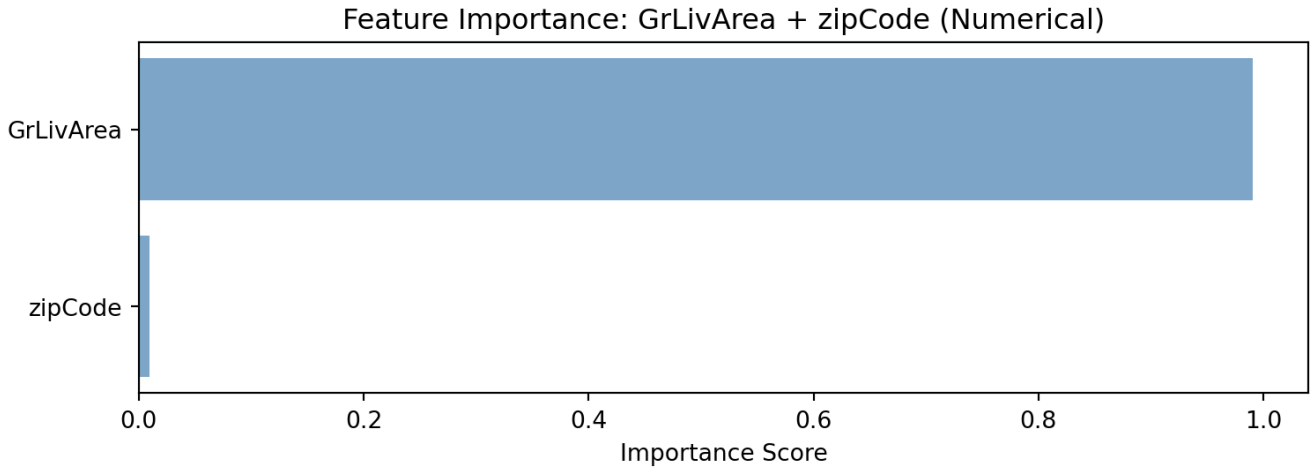
In the full model above, `zipCode` competes against 8 other strong numerical predictors and never gets chosen for a split. That makes it hard to tell whether zip code is truly unimportant or whether the encoding is hiding its value. Let's isolate the effect by building a simplified model that uses **only** `GrLivArea` and `zipCode` as predictors, treating zip code as numerical.

Simplified Model: GrLivArea + zipCode (Numerical)

```python
X_simple_num = model_data[['GrLivArea', 'zipCode']]
y_simple = model_data['SalePrice']

X_train_sn, X_test_sn, y_train_sn, y_test_sn = train_test_split(
    X_simple_num, y_simple, test_size=0.2, random_state=123)

tree_simple_num = DecisionTreeRegressor(
    max_depth=3, min_samples_split=20, min_samples_leaf=10, random_state=123)
tree_simple_num.fit(X_train_sn, y_train_sn)

r2_num = r2_score(y_test_sn, tree_simple_num.predict(X_test_sn))
print(f"Test R² (numerical zipCode): {r2_num:.4f}")
```

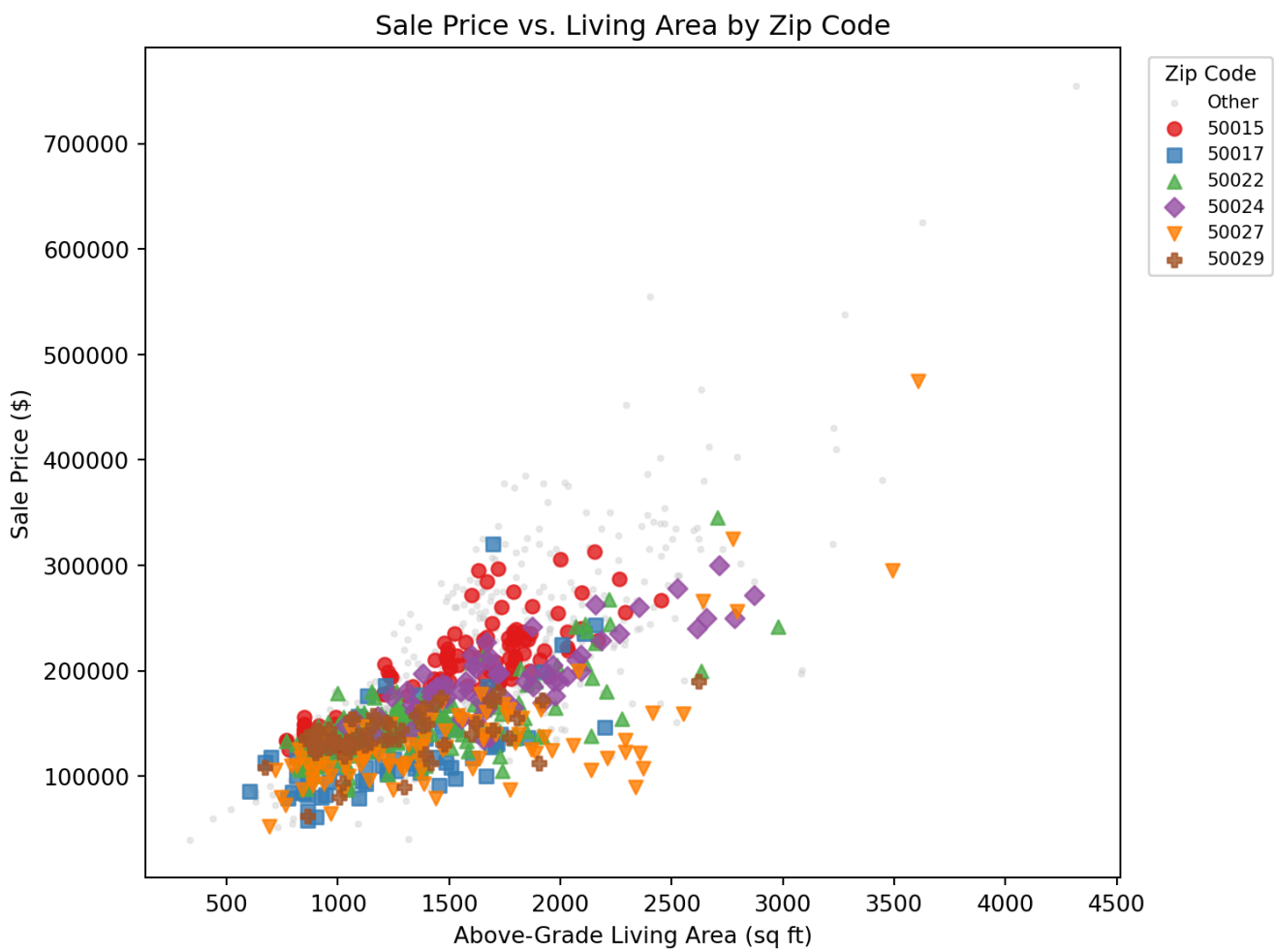
```
Test R² (numerical zipCode): 0.4502
```





Visualizing the Zip Code Effect

The tree above ignores zip code entirely – every split is on `GrLivArea`. But does zip code actually matter for house prices? Before trying a different encoding, let's look at the raw data. If zip codes don't separate prices, there's nothing to fix. If they do, the numerical encoding is hiding real signal.



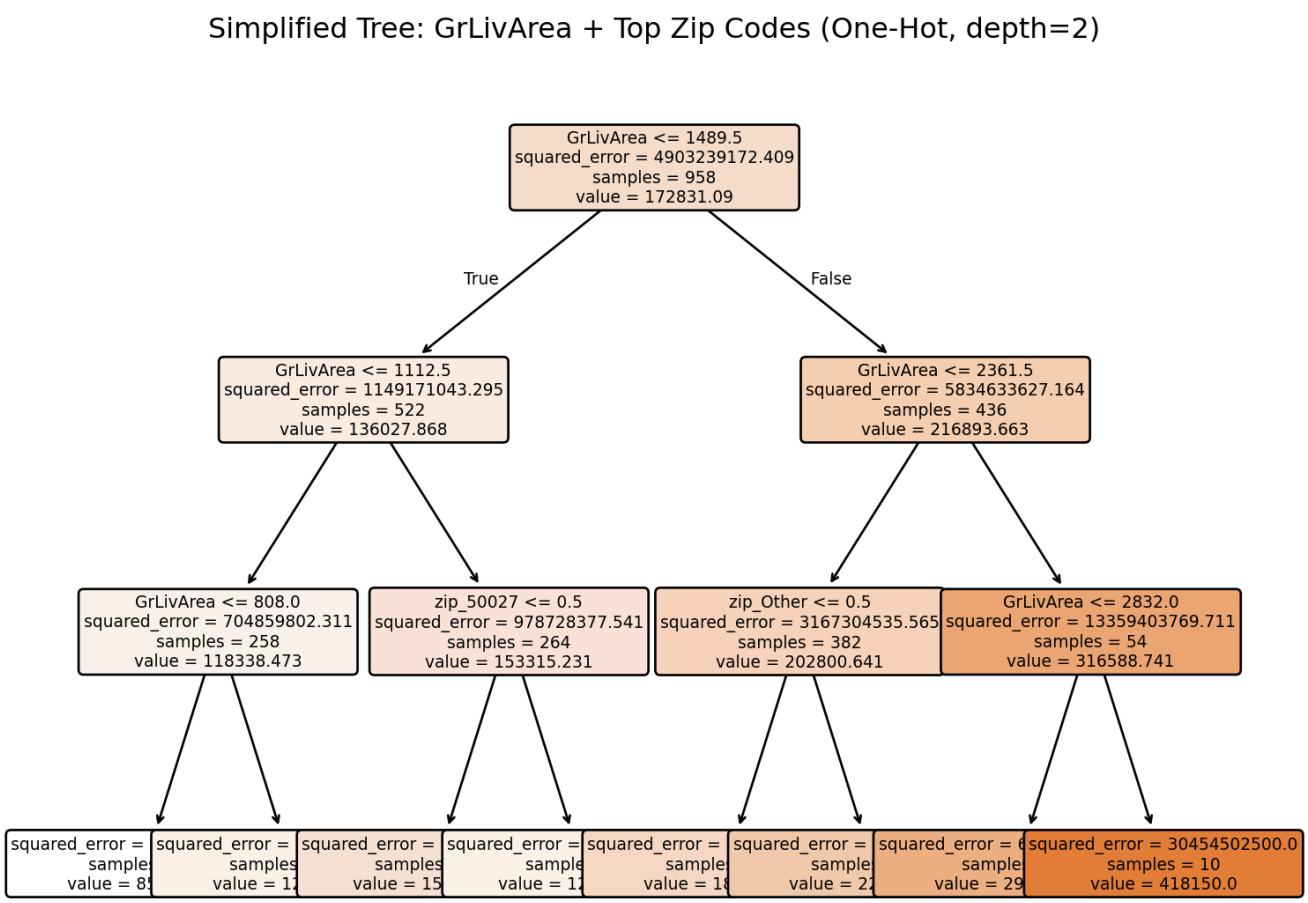
The scatter plot makes the case visually: different zip codes cluster at different price levels, even for houses with the same living area. A 1,500 sq ft house in one zip code can sell for dramatically more than the same-sized house in another. Zip code clearly matters – so why does the numerical tree ignore it?

The answer is encoding. A numerical split like “zipCode > 50012.5” has no meaningful relationship to price. To let the tree see zip code differences, we can one-hot encode the top zip codes and rebuild the model.

One-Hot Encoding the Top Zip Codes

```
zip_dummies = pd.get_dummies(
    model_data['zipCode'].apply(lambda z: str(z) if z in top_zips else 'Other'),
    prefix='zip')
X_onehot = pd.concat([model_data[['GrLivArea']], zip_dummies], axis=1)

X_train_oh, X_test_oh, y_train_oh, y_test_oh = train_test_split(
    X_onehot, y_simple, test_size=0.2, random_state=123)

tree_onehot = DecisionTreeRegressor(
    max_depth=3, min_samples_split=20, min_samples_leaf=10, random_state=123)
tree_onehot.fit(X_train_oh, y_train_oh)

r2_oh = r2_score(y_test_oh, tree_onehot.predict(X_test_oh))
print(f"Test R² (one-hot top zips, depth=3): {r2_oh:.4f}")
```

```
Test R² (one-hot top zips, depth=3): 0.4249
```



What the One-Hot Tree Reveals

With one-hot encoding, the tree can now ask "is this house in zip code 50027?" – and it does. The tree isolates specific zip codes as meaningful predictors, something the numerical encoding could never achieve. Notice how zip code 50027 appears as a split that identifies a lower-priced neighborhood cluster.

However, one-hot encoding has its own limitations: each zip code becomes its own binary column, and the tree can only ask about one zip code at a time per split. A truly categorical-aware algorithm could instead split on *subsets* of zip codes in a single decision – for example, {50022, 50015} vs. {50027, 50013, ...} – capturing richer groupings. That is the frontier explored in Question 3 below.

Discussion Questions for Challenge

Your Task: Add thoughtful narrative answers to these questions in the Discussion Questions section of your rendered HTML site.

1. **Numerical vs Categorical Encoding:** In the simplified model above, zip code is treated as a numerical variable. Look at the tree and the scatter plot. Given what you know about zip codes and real estate prices, how should zip code be modelled, numerically or categorically? Is zipcode an ordinal or non-ordinal variable?

2. **R vs Python Implementation Differences:** When modelling zip code as a categorical variable, the output tree and feature importance would differ quite significantly had you used R as opposed to Python. Investigate why this is the case. What does R offer that Python does not? Which language would you say does a better job of modelling zip code as a categorical variable? Can you quote the documentation at https://scikit-learn.org/stable/modules/tree.html suggesting a weakness in the Python implementation? If so, please provide a quote from the documentation.

3. **Can You Get a Decision Tree in Python That Splits on Subsets of Zip Codes?** Find a Python package that handles categorical variables natively (i.e. not one-hot encoding) and re-run the simplified model (`GrLivArea` + `zipCode`) using it. Does the resulting tree split on *clusters* or *subsets* of zip codes rather than numerical thresholds? Show the tree and discuss what you find.

⚠️ **Fair Warning on Question 3**

This last question is worth only 5 points (the difference between 95% and 100%). Finding and getting a new package to work can be time-consuming and frustrating – especially when documentation is sparse and AI assistants have limited knowledge of newer libraries. **Be prepared to abort this attempt if it's eating too much of your time.** A solid 95% is an excellent grade; don't let a 5-point stretch goal turn into a multi-hour rabbit hole.

Submission Checklist ✅

- [ ] Forked starter repository from https://github.com/flyaflya/decTreeChallenge
- [ ] Cloned repository locally using Cursor (or VS Code)
- [ ] Analysis written in `index.qmd` with clear Q&A narrative
- [ ] Document rendered to HTML successfully
- [ ] HTML committed and pushed to your forked repository
- [ ] GitHub Pages enabled (Deploy from a branch → main → /(root))
- [ ] Site accessible at `https://[your-username].github.io/decTreeChallenge/`